

HBalloc: A High-performance Persistent Memory Allocator via Non-blocking Global Allocation and Batched Persistence

Yujie Hu, Yu Hua, Xiangyu Xiang, Aoyang Tong
Huazhong University of Science and Technology
Xue Liu, *McGill University*

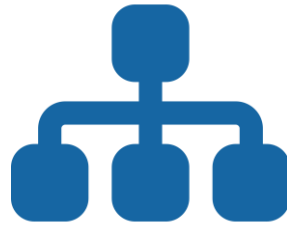
CASES 2026

Background

File systems



Index structures



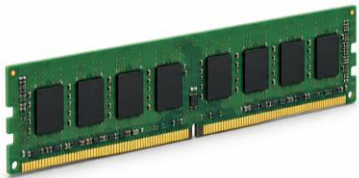
Development
libraries



In-memory DBs



Memory and storage media



DRAM
volatile



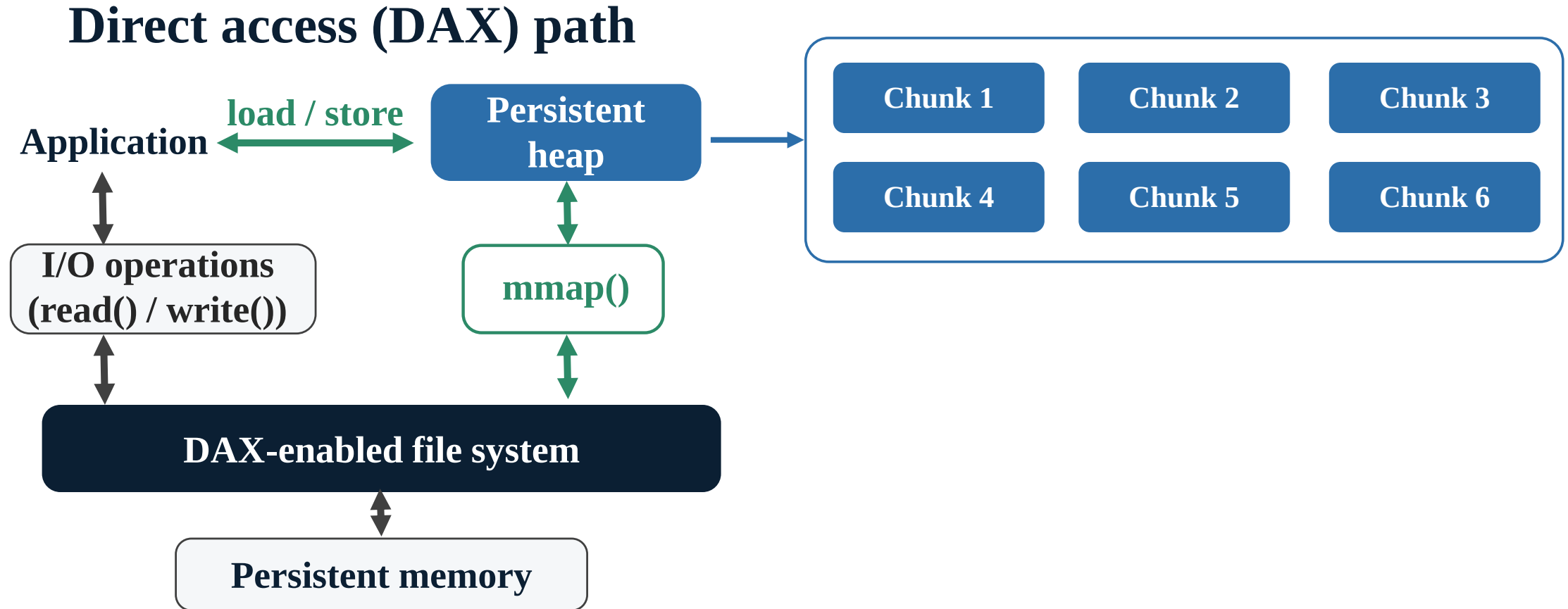
Persistent Memory
durable + byte-addressable



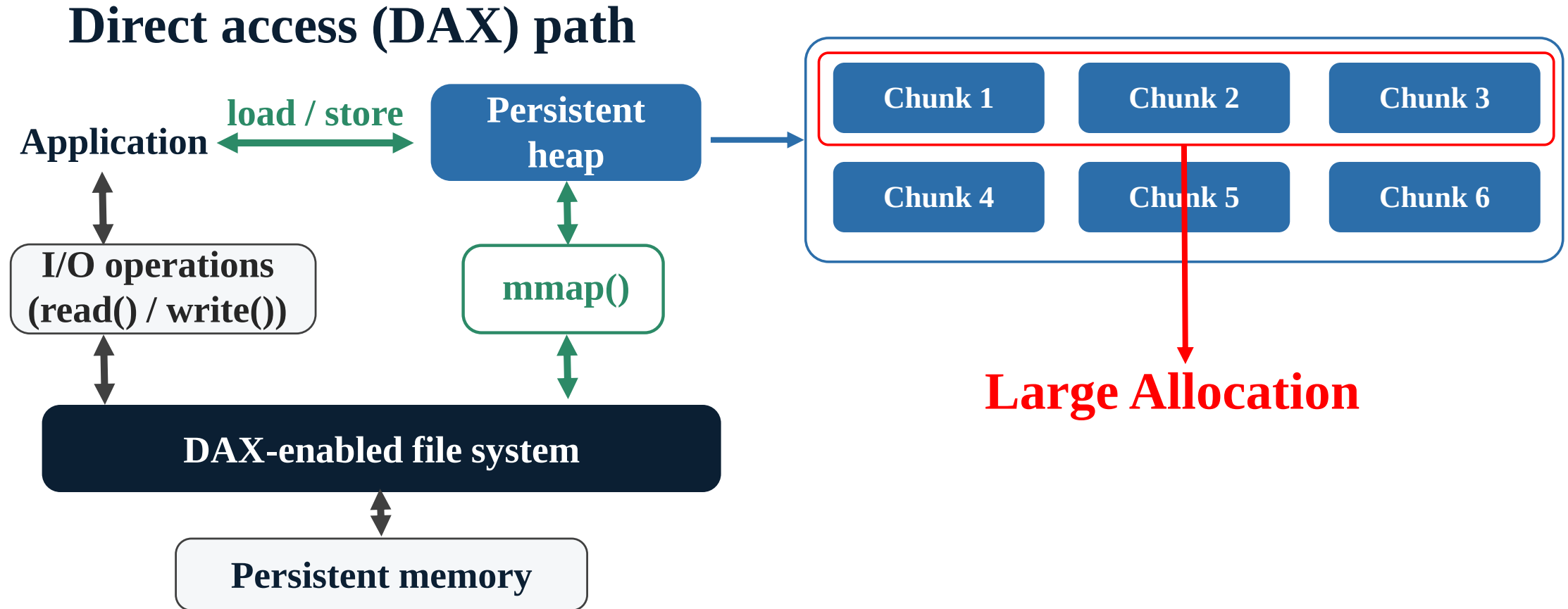
SSD
block access

Persistency, byte-addressability, near-DRAM latency, large capacity

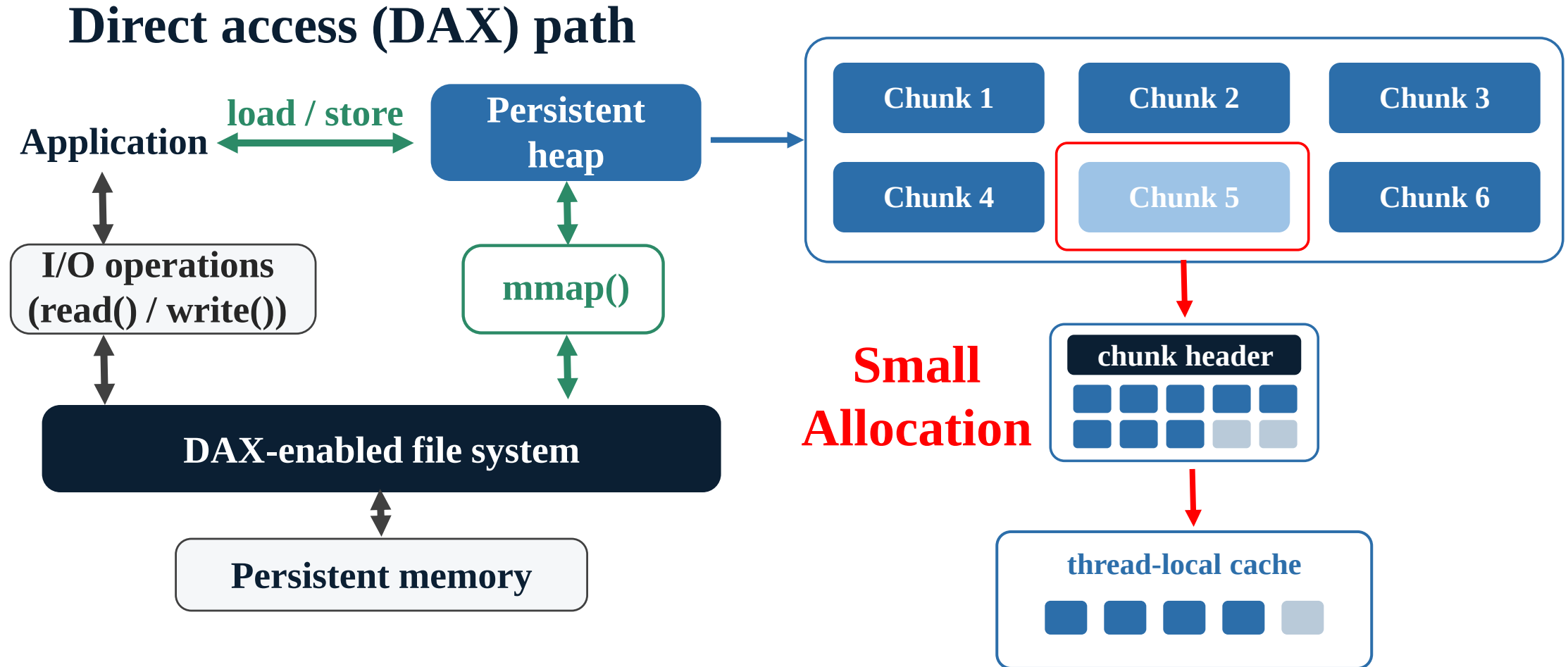
Persist Memory Allocation



Persist Memory Allocation

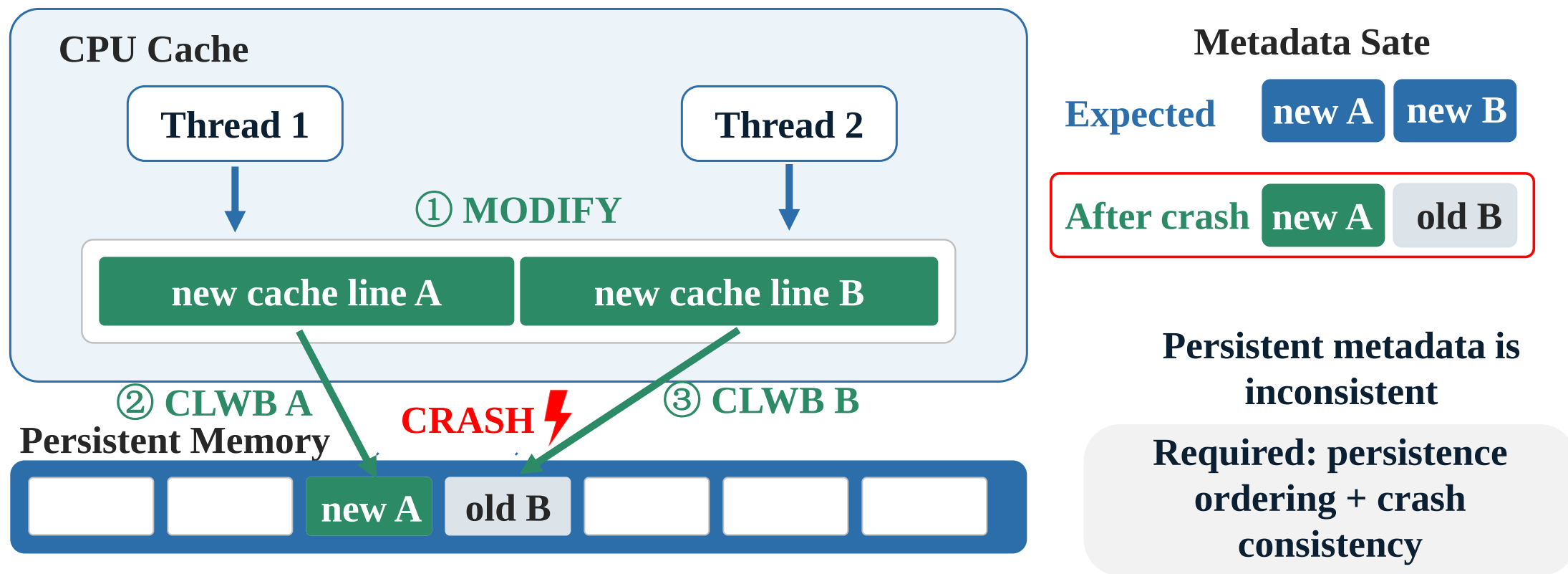


Persist Memory Allocation



Memory Allocator

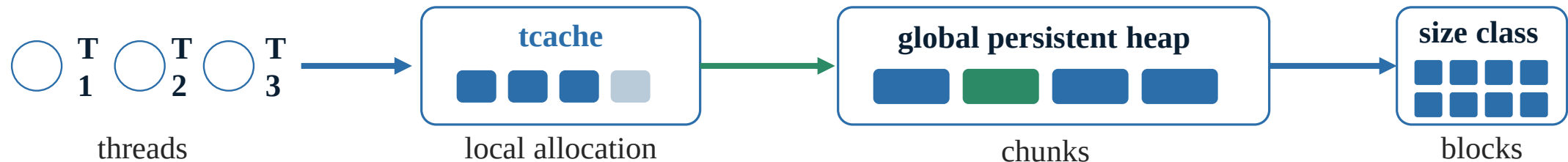
- A crash between two cache-line write-backs results in *partial updates* upon persistent metadata



Persistent Memory Allocator

➤ PM allocators reuse heap structures with persistence support

Persistent Heap Management



Crash-Consistency Models

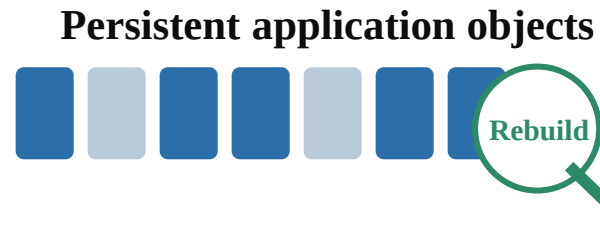
Write-ahead Logging



nvm_malloc (VLDB 2015),
Pallocator (PVLDB 2017),
NValloc-LOG (ASPLOS 2022),
PMA (ICCD 2023)

Strong consistency

Post-crash Garbage Collection



Makalu (ACM SIGPLAN Notices 2016),
ralloc (ISMM 2020),
NValloc-GC (ASPLOS 2023)

Slow recovery

Internal Collection



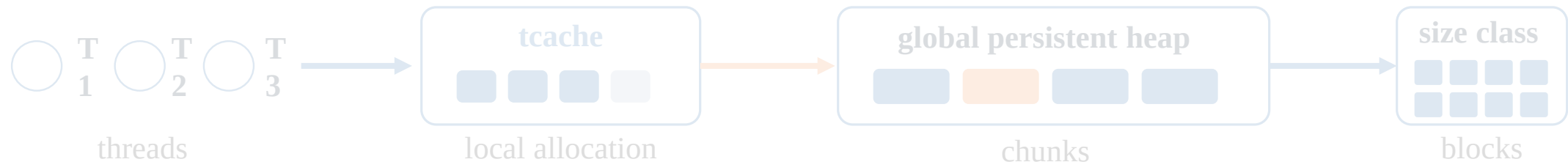
PMDK (Intel)

High flush latency

Persistent Memory Allocator

- PM allocators reuse heap structures with persistence support

Persistent Heap Management



How can a PM allocator achieve high throughput for large allocation while keeping pointers valid after object relocation?

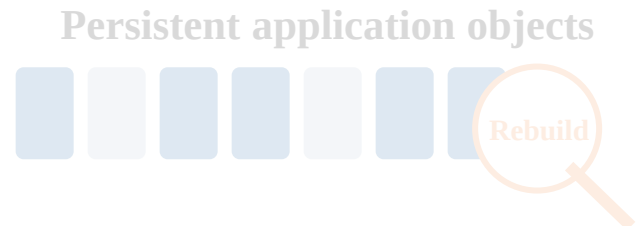
Crash-Con

Write-ahead Logging



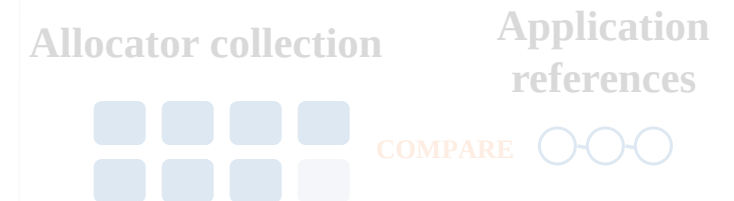
nvm_malloc (VLDB 2015),
Pallocator (PVLDB 2017),
NValloc-LOG (ASPLOS 2022),
PMA (ICCD 2023)

Post-Crash Garbage Collection



Makalu (ACM SIGPLAN Notices 2016),
ralloc (ISMM 2020),
NValloc-GC (ASPLOS 2023)

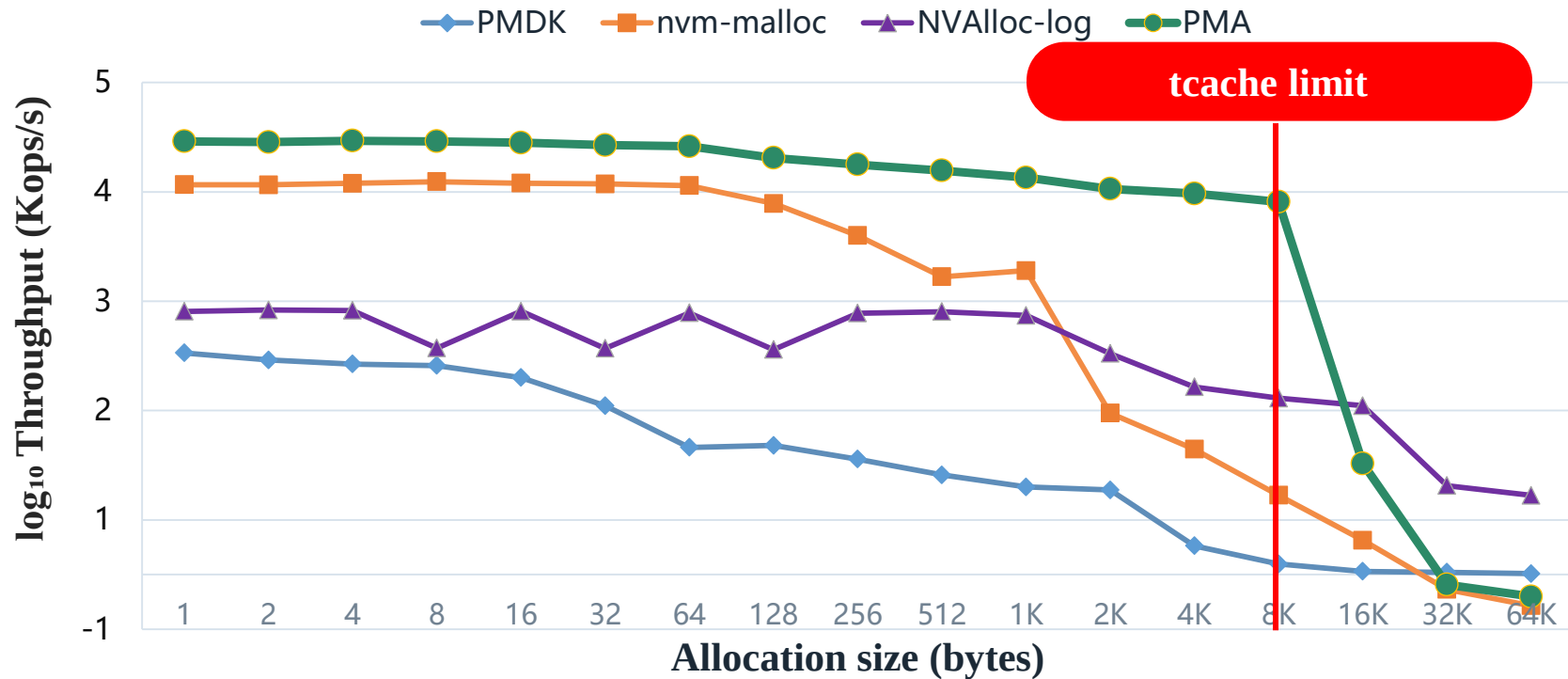
Internal Collection



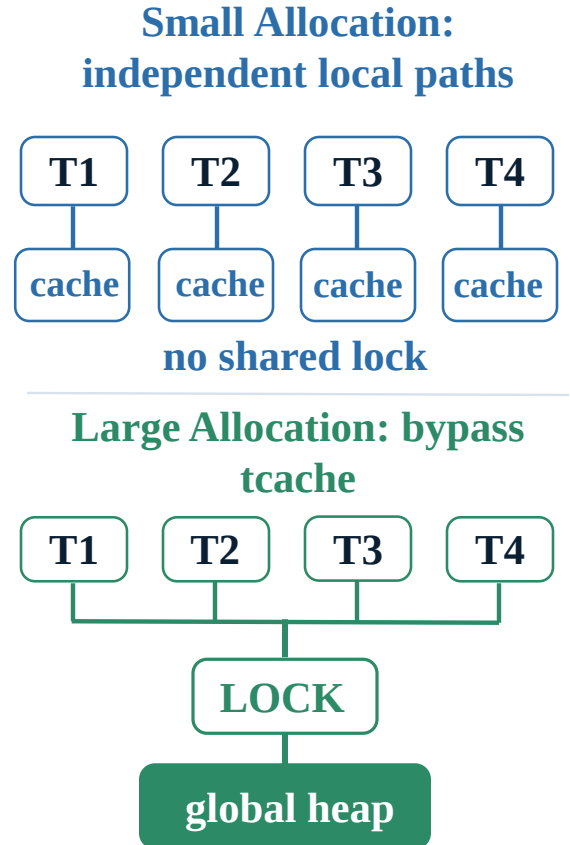
PMDK (Intel)

Challenge 1: Allocation Size Sensitivity

- Existing PM allocators drop in throughput as objects reach *tcache* limit



PMA: 2,563 → 0.63 Kops/s
8 KB to 64 KB: 4,069× lower

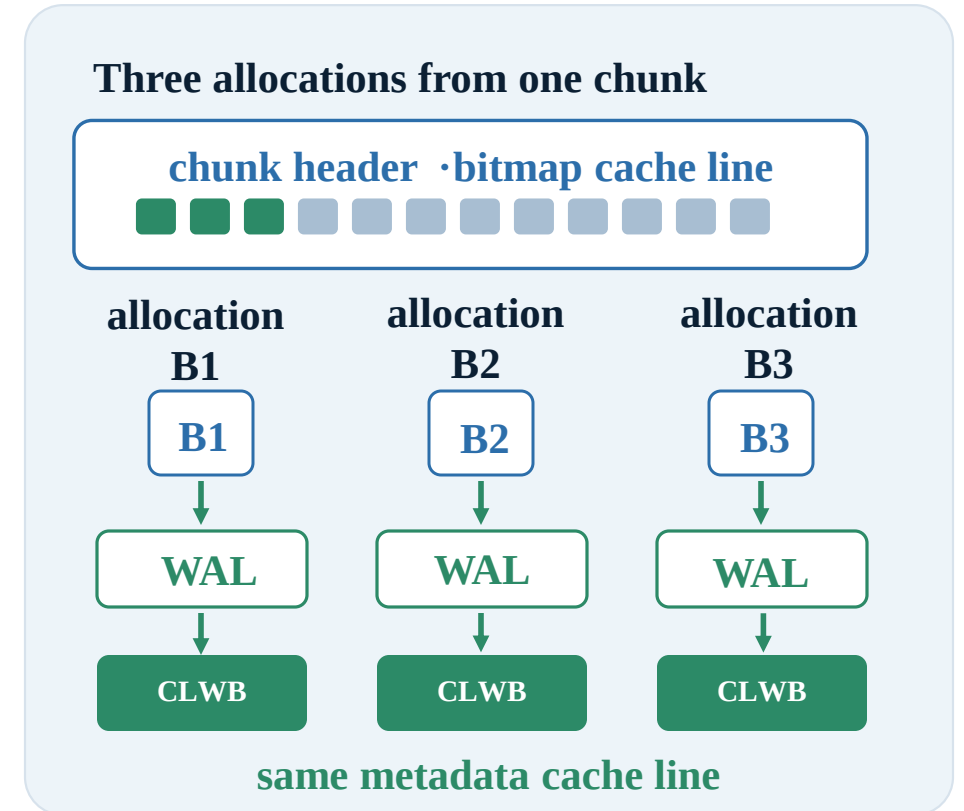
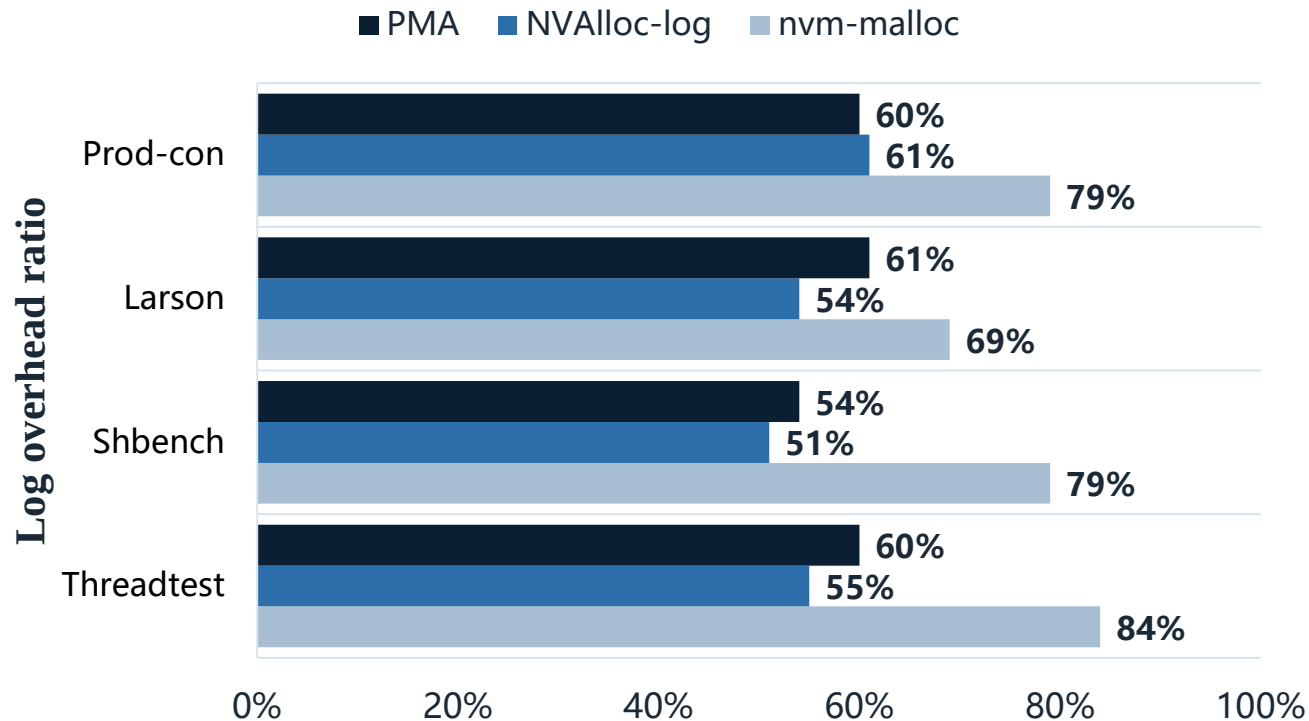


Large requests are blocked on lock-protected global heap

Challenge 2: High Persistence Tax

➤ Small allocations repeatedly log and flush the same chunk-header

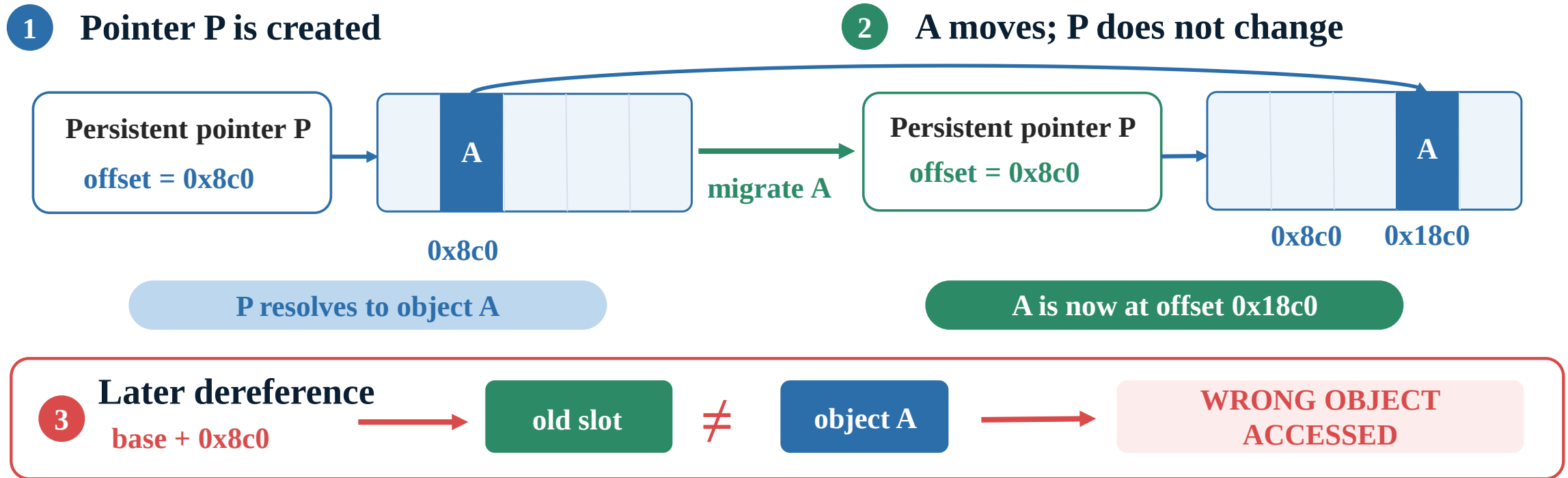
Log overhead for small allocations



WAL and repeated CLWB dominate the cost of small allocations

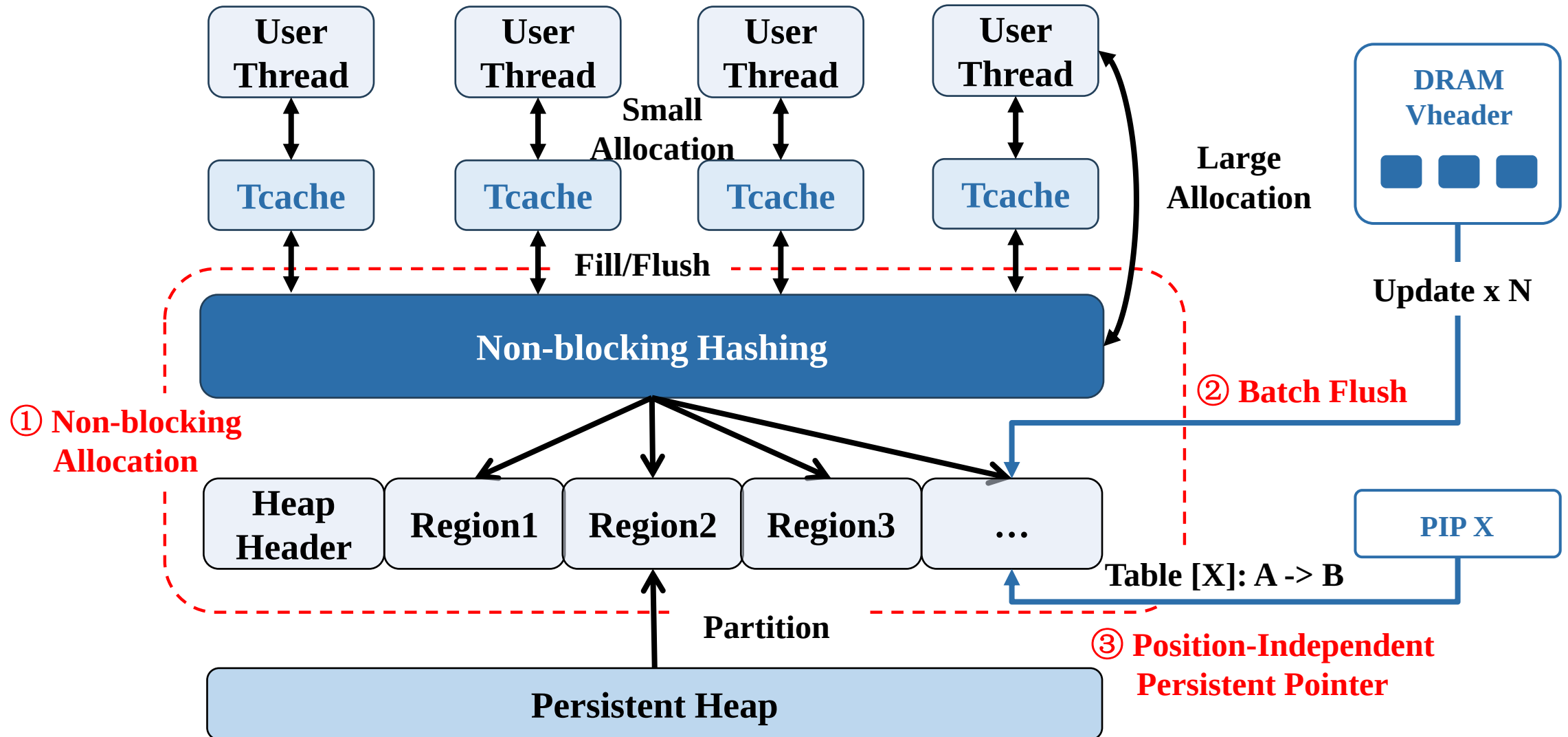
Challenge 3: Location-Dependent Pointers

➤ After object migration, existing offset pointers still reference the old location



Existing references cannot follow the moved object

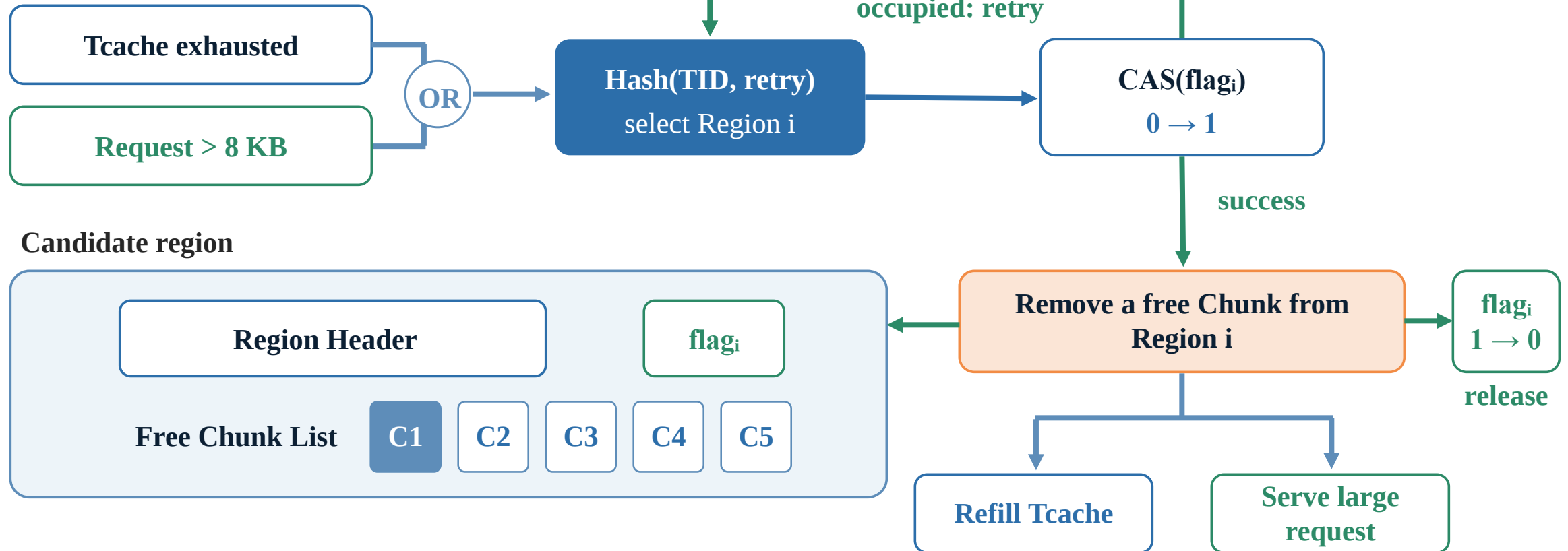
Hballoc: Overview



Non-Blocking Global Allocation

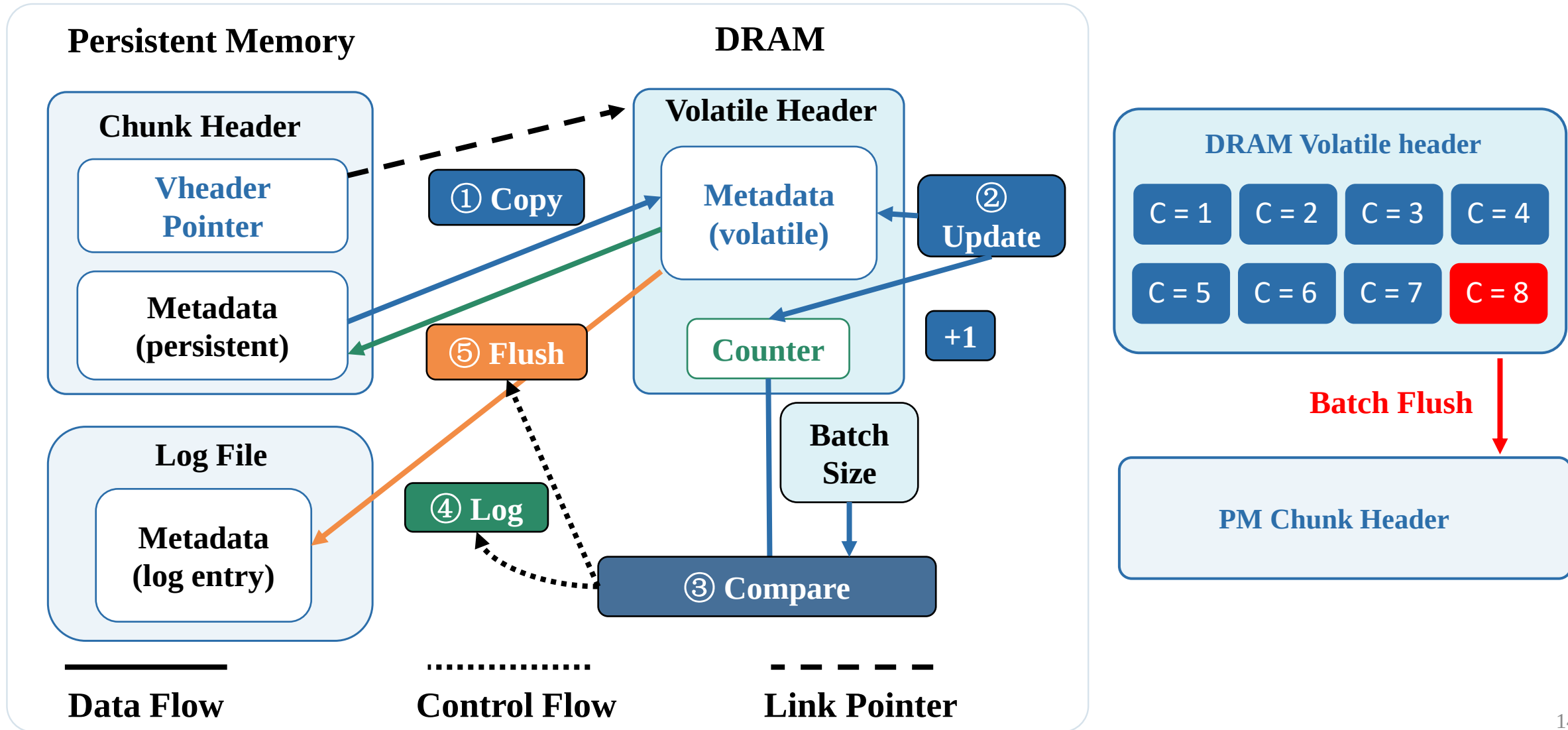
- Hash each global request to an independent *Region*, which is acquired with CAS

Global allocation is triggered by



Batched Persistence

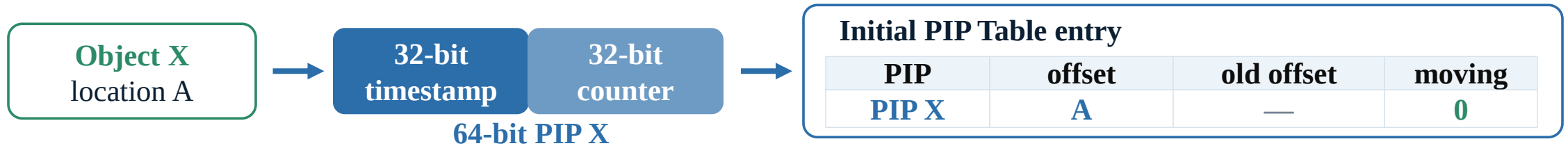
- Update allocator metadata in DRAM and persist in batches



Position-Independent Persistent Pointer

➤ A stable PIP keeps references valid when an object moves

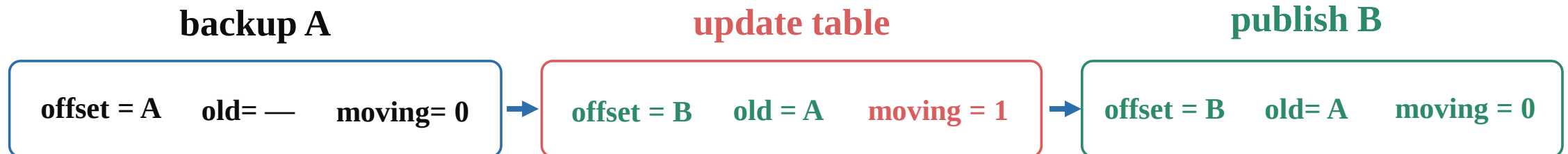
1 Allocate and register Object X



2 Dereference PIP X



3 Move Object X: A → B



Performance Evaluation Setup

➤ **Testbed**

- 2 × Intel Xeon Gold 6230R @ 2.10 GHz, 26 cores per socket, 6 × 128 GB Intel Optane DC PMM, 12 × 16 GB DDR4, Ubuntu 18.04

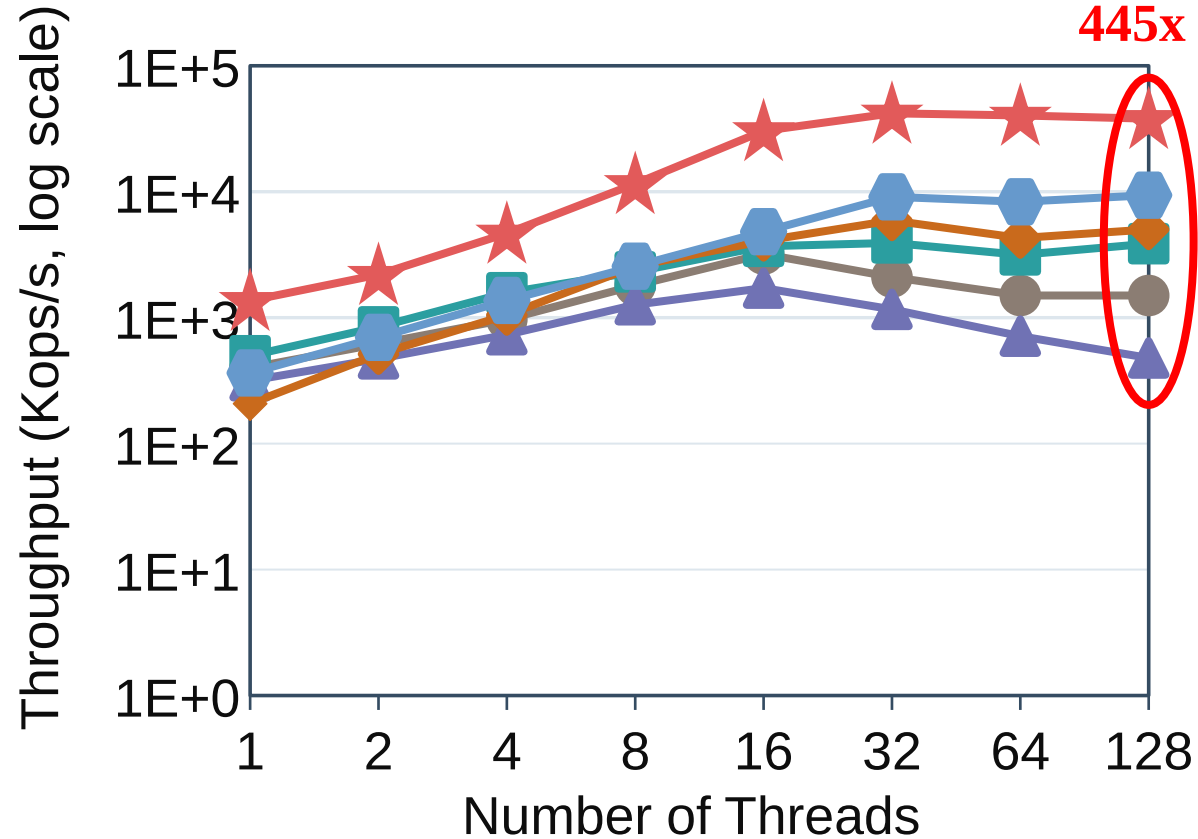
➤ **Benchmarks**

- Threadtest, Shbench, Larson, Prod-con, FPTree

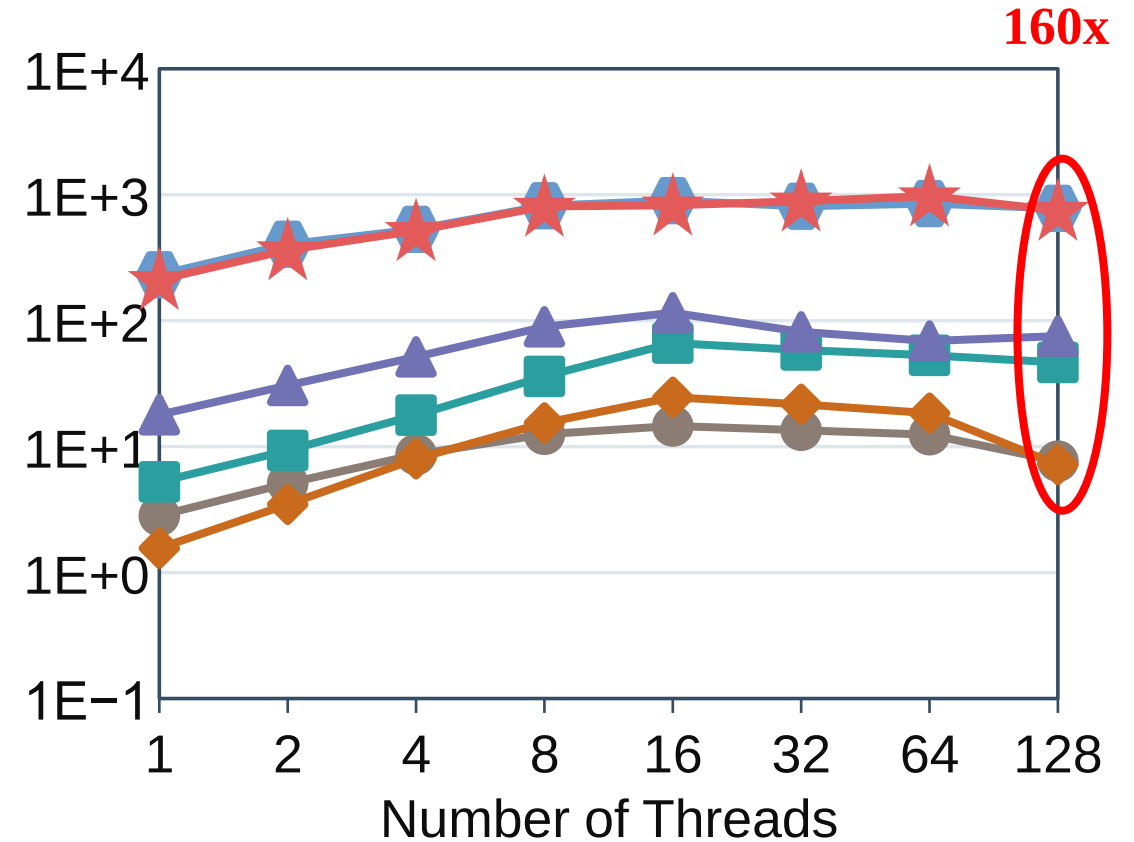
➤ **Competitors**

- PMDK (Intel), nvm_malloc (VLDB 2015), NVAlloc-log (ASPLOS 2022), PMA (ICCD 2023)

Allocation Throughputs

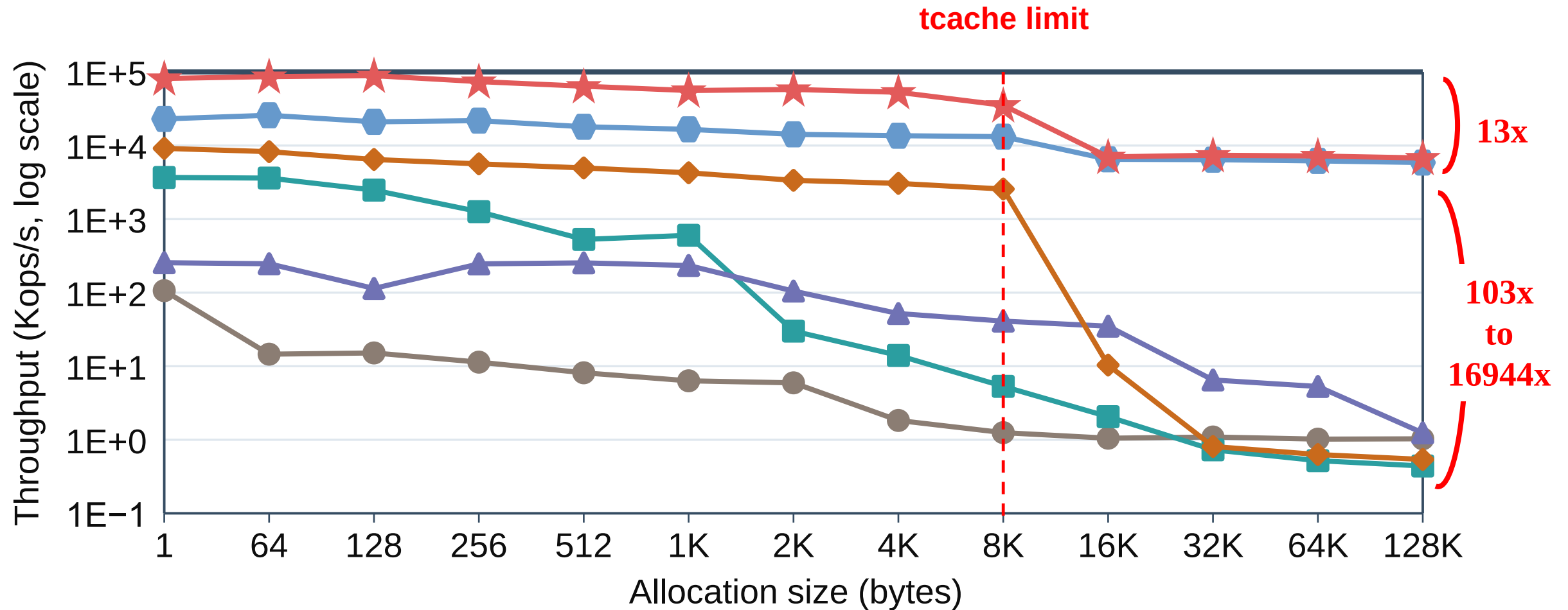


Larson · small allocations



Prod-con · large allocations

Allocation Sizes



Summary

- Existing PM allocators suffer from **global lock contention**, **repeated persistence overhead**, and **location-dependent pointers**.
- HBalloc addresses these bottlenecks with three designs:
 - **Non-blocking global allocation**
 - **Batched persistence**
 - **Position-independent persistent pointers**
- HBalloc improves throughput by up to **445×** for small allocations and **160×** for large allocations.

Thanks Q&A

Open-source code: github.com/HUSTyjhu/HBalloc